

Universidad de San Carlos de Guatemala  
Centro Universitario de Occidente  
División de Ciencias de la Ingeniería  
Ingeniería en Ciencias y Sistemas

# Práctica 1

(Manual Técnico)

**Compiladores 2**

**REGISTRO:** 202030849

**NOMBRE:** María Monserrat Gómez Rabatú

**ID:** 4

*24 de agosto, 2026*

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>Introducción</b>                             | <b>2</b>  |
| Propósito del documento                         | 2         |
| Alcance del proyecto                            | 2         |
| Tecnologías y herramientas utilizadas           | 3         |
| com.compi2.codexlatinus - Núcleo del compilador | 4         |
| com.compi2.codexlatinus.ast                     | 4         |
| com.compi2.codexlatinus.ast.expresiones         | 4         |
| com.compi2.codexlatinus.ast.instrucciones       | 5         |
| com.compi2.codexlatinus.errores                 | 5         |
| com.compi2.codexlatinus.gui / gui.controllers   | 6         |
| com.compi2.codexlatinus.pila                    | 6         |
| com.compi2.codexlatinus.semantico               | 6         |
| com.compi2.codexlatinus.visitor                 | 7         |
| <b>Análisis Léxico</b>                          | <b>7</b>  |
| Definición de tokens                            | 7         |
| Detección y manejo de errores léxicos           | 10        |
| <b>Análisis Sintáctico</b>                      | <b>10</b> |
| Generación del parser con ANTLR4                | 10        |
| Reglas principales de la gramática              | 10        |
| Manejo de errores sintácticos                   | 11        |
| Pila de análisis sintáctico                     | 11        |

# Introducción

## Propósito del documento

El presente documento tiene como finalidad describir de manera técnica el diseño, la arquitectura y la implementación del compilador para el lenguaje CodexLatinus. Se detallan las decisiones de diseño tomadas en cada una de las fases del proceso de compilación (análisis léxico, sintáctico y semántico), así como la arquitectura de la interfaz gráfica desarrollada para interactuar con el compilador.

## Alcance del proyecto

El proyecto consiste en el desarrollo de un compilador para CodexLatinus, un lenguaje de programación de propósito educativo con sintaxis basada en palabras del latín. El sistema implementado abarca:

- Un analizador léxico y sintáctico generado mediante ANTLR4 a partir de una gramática formal definida para el lenguaje.
- Un analizador semántico encargado de validar tipos de datos, ámbitos (scopes) y uso correcto de variables, funciones y estructuras.
- La construcción de un Árbol de Sintaxis Abstracta (AST) para representar el programa analizado.
- Un editor de código integrado con resaltado de sintaxis en tiempo real, basado en el mismo analizador léxico del compilador.
- Herramientas de visualización para el AST, la tabla de símbolos y la pila de análisis sintáctico.
- Una funcionalidad adicional de traducción del código fuente a PigLatin.

El proyecto no contempla la generación de código ejecutable ni la interpretación/ejecución del programa analizado; su alcance se limita a las fases de análisis (léxico, sintáctico y semántico) y a las herramientas de apoyo visual antes mencionadas.

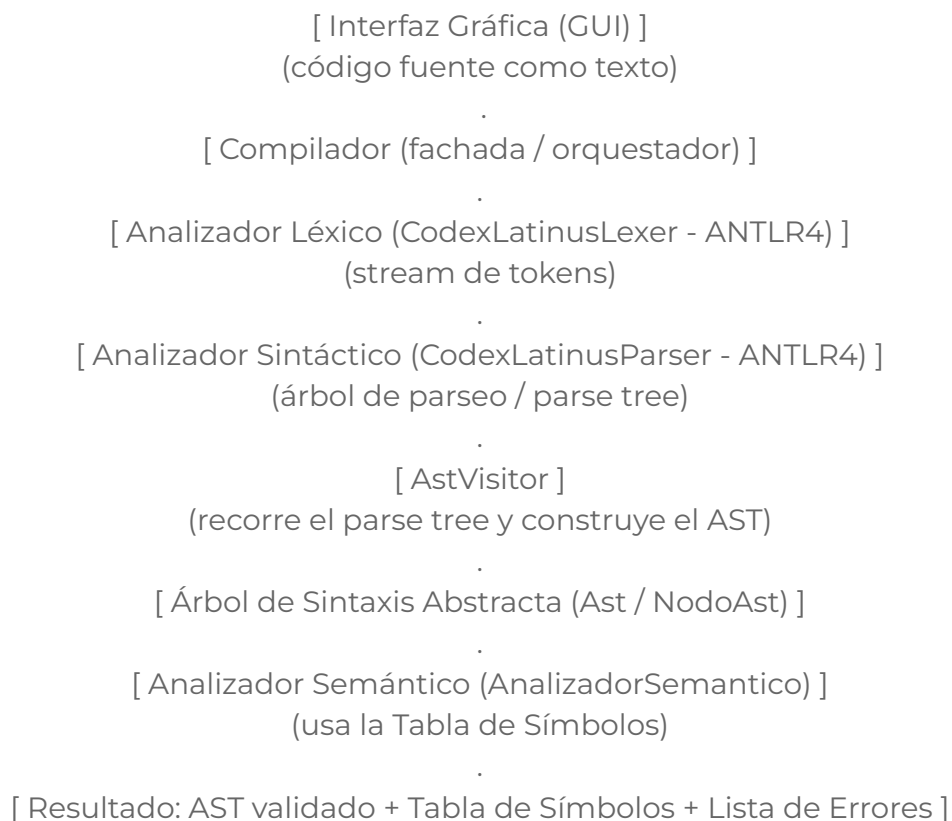
## Tecnologías y herramientas utilizadas

| Herramienta / Tecnología | Uso dentro del proyecto                                                                                |
|--------------------------|--------------------------------------------------------------------------------------------------------|
| Java                     | Lenguaje de desarrollo del compilador y la aplicación                                                  |
| JavaFX                   | Framework para la interfaz gráfica de usuario                                                          |
| ANTLR4                   | Generación del analizador léxico y sintáctico a partir de la gramática formal                          |
| RichTextFX               | Componente de editor de texto (CodeArea) con soporte para resaltado de sintaxis y numeración de líneas |
| Maven                    | Gestión de dependencias y construcción del proyecto                                                    |

## 2. Arquitectura General del Sistema

### 2.1 Visión general

El sistema está organizado en capas claramente separadas, siguiendo el flujo estándar de un compilador:



La clase `Compilador` actúa como fachada (patrón *Facade*): centraliza el flujo completo de análisis (`analizar()`) y expone además un punto de entrada independiente para el análisis puramente léxico (`tokenizar()`), utilizado por el editor para el resaltado de sintaxis en tiempo real sin necesidad de ejecutar el análisis sintáctico ni semántico completo.

## Estructura de paquetes

### `com.compi2.codexlatinus`

- **Compilador:** fachada principal; expone `analizar()` (léxico + sintáctico + semántico) y `tokenizar()` (solo léxico, para resaltado en tiempo real).
- **ArbolBuilder:** clase de apoyo encargada de construir o ensamblar la representación gráfica/visual del AST a partir de los nodos generados por `AstVisitor`, para su posterior renderizado en la ventana de visualización.
- **NodoGrafico:** representa un nodo del AST adaptado para su despliegue gráfico (posición, forma, conexiones con nodos hijos), usado por `AstController` para dibujar el árbol en pantalla.
- **Tipo:** representa un tipo de dato del lenguaje (`numerus`, `textum`, `decimalis`, `littera`, `bool`, o un tipo de estructura definida por el usuario), usado durante la validación semántica de tipos.
- **TipoEnum:** enumeración que lista las categorías de tipos soportadas por el lenguaje, utilizada internamente por `Tipo` para clasificar y comparar tipos de forma segura.
- **TraductorPigLatin:** traduce el código fuente (o los identificadores/palabras del programa) a PigLatin, como funcionalidad adicional del proyecto.

### `com.compi2.codexlatinus.ast`

- **Ast:** representa el árbol de sintaxis abstracta completo (raíz del programa analizado).
- **NodoAst:** clase base/abstracta de la que heredan todos los tipos de nodos del AST (instrucciones y expresiones).

### `com.compi2.codexlatinus.ast.expresiones`

Contiene las clases que representan expresiones del lenguaje:

- **Expresion:** clase base abstracta para todo nodo de expresión.
- **AccesoVariable:** representa el acceso a una variable, incluyendo accesos a atributos (`.`) y a elementos de arreglo (`[]`).
- **Literal:** representa un valor literal (número, decimal, cadena, carácter, booleano).
- **LlamadaFuncion:** representa la invocación de una función dentro de una expresión.
- **Operacion:** representa una operación binaria o unaria (aritmética, relacional, lógica).

- **ParteAcceso:** representa un segmento individual dentro de un acceso encadenado (por ejemplo, cada `.atributo` o `[índice]` en `objeto.atributo[i]`).

### **com.compi2.codexlatinus.ast.instrucciones**

Contiene las clases que representan instrucciones del lenguaje:

- **Instruccion:** clase base abstracta para todo nodo de instrucción.
- **Declaracion:** representa la declaración de una variable simple.
- **DeclaracionArray:** representa la declaración de un arreglo (series).
- **DeclaracionStructura:** representa la definición o el uso de una estructura (structura).
- **Asignacion:** representa la asignación de un valor a una variable o atributo existente.
- **AsignacionAtributo / Atributo:** representan la asignación y definición de atributos dentro de una estructura.
- **Actualizacion:** representa operaciones de incremento/decremento (`++`, `--`) sobre una variable.
- **Condicional / RamaCondicional:** representan la estructura de control `si/aliter/finis` y cada una de sus ramas.
- **CicloDum, CicloFacere, CicloPer:** representan los ciclos `dum`, `facere...dum` y `per`, respectivamente.
- **Perge / Interrumpe:** representan las instrucciones de control de flujo `continue` y `break` del lenguaje.
- **Funcion:** representa la definición de una función (con o sin retorno).
- **Parametro:** representa un parámetro formal de una función.
- **Retorno:** representa la instrucción `reddere` (retorno de valor).
- **LlamadaFuncionInst:** representa una llamada a función usada como instrucción independiente (no como parte de una expresión).
- **Lectura / Escritura:** representan las instrucciones de entrada y salida del lenguaje.
- **Finis / Structura:** nodos de apoyo relacionados con el cierre de bloques y la representación de estructuras definidas.

### **com.compi2.codexlatinus.errorres**

- **Error:** representa un error (léxico, sintáctico o semántico) detectado durante el análisis, encapsulando su mensaje y ubicación (línea/columna).
- **ErrorListener:** listener personalizado que captura los errores reportados por ANTLR durante el análisis y los transforma en instancias de **Error** para ser mostrados en la interfaz.

## **com.compi2.codexlatinus.gui / gui.controllers**

- MainController: controlador principal de la ventana del editor; gestiona el área de código, la ejecución del análisis y la apertura de las vistas secundarias.
- Resaltador: encargado del resaltado de sintaxis en tiempo real, reutilizando el lexer (CodexLatinusLexer) del propio compilador.
- AstController: controlador de la ventana de visualización del AST.
- PilaController: controlador de la ventana de visualización de la pila de análisis sintáctico.
- PigLatinController: controlador de la ventana que muestra la traducción a PigLatin.
- TablaSimboloController: controlador de la ventana que muestra la tabla de símbolos.
- Rutas: clase de constantes que centraliza las rutas de los recursos de la aplicación (archivos FXML y hojas de estilo CSS).
- EstilosController: controlador encargado de aplicar o gestionar los estilos visuales (temas/CSS) de la aplicación.
- VentanaUtil: clase utilitaria con métodos de apoyo para la creación y configuración de nuevas ventanas (Stage) de la aplicación.

## **com.compi2.codexlatinus.pila**

- EstadoPila: representa una instantánea del estado de la pila del analizador sintáctico en un punto determinado del proceso de parseo (símbolos apilados, entrada restante, acción aplicada).
- GeneradorPila: genera la secuencia completa de estados de la pila durante el análisis sintáctico, para su posterior visualización paso a paso.

## **com.compi2.codexlatinus.semantico**

- TablaSimbolos: estructura que almacena los símbolos (variables, funciones, estructuras) declarados en el programa, organizados por ámbito.
- Simbolo: representa un símbolo individual, con atributos como nombre, tipo y ámbito de declaración.
- Scope: representa un ámbito de visibilidad (global, de función o de bloque), permitiendo anidamiento y búsqueda de símbolos.
- TipoSimbolo: enumeración o clasificación que indica la naturaleza del símbolo (variable, función, parámetro, estructura).
- AnalizadorSemantico: recorre el AST y aplica las reglas de validación semántica (tipos compatibles, variables declaradas antes de su uso, duplicidad de identificadores, etc.).

## com.compi2.codexlatinus.visitor

- AstVisitor: extiende el visitor generado por ANTLR (CodexLatinusBaseVisitor); recorre el árbol de parseo y construye el AST correspondiente.
- BuilderTablaSimbolos: construye la tabla de símbolos a partir del recorrido del AST, registrando cada declaración en el ámbito correspondiente.
- AgregarStmt: clase de apoyo utilizada durante la construcción del AST para añadir instrucciones (statements) a los bloques correspondientes (cuerpos de funciones, ciclos, condicionales).

## Análisis Léxico

### Definición de tokens

El analizador léxico fue generado con ANTLR4 a partir de las reglas léxicas definidas en la gramática **CodexLatinus.g4**. La siguiente tabla resume los tokens reconocidos por el lexer:

| Token      | Patrón                 | Categoría                                                      |
|------------|------------------------|----------------------------------------------------------------|
| ESTO       | <code>esto</code>      | Palabra reservada (declaración de variable)                    |
| SERIES     | <code>series</code>    | Palabra reservada (declaración de arreglo)                     |
| STRUCTURA  | <code>structura</code> | Palabra reservada (definición de estructura)                   |
| FINIS      | <code>finis</code>     | Palabra reservada (cierre de bloque: si/dum/structura/función) |
| FINIS_PROG | <code>FINIS</code>     | Palabra reservada (cierre de la sección MAIOR>)                |
| SI         | <code>si</code>        | Palabra reservada (condicional)                                |
| ALITER     | <code>aliter</code>    | Palabra reservada (rama alternativa del condicional)           |
| DUM        | <code>dum</code>       | Palabra reservada (ciclo mientras)                             |
| FACERE     | <code>facere</code>    | Palabra reservada (ciclo hacer-mientras)                       |
| PER        | <code>per</code>       | Palabra reservada (ciclo for)                                  |



|                                      |                                    |                                                |
|--------------------------------------|------------------------------------|------------------------------------------------|
| PERGE                                | <code>perge</code>                 | Palabra reservada (continue)                   |
| INTERRUMPE                           | <code>interrumpe</code>            | Palabra reservada (break)                      |
| RATIO                                | <code>ratio</code>                 | Palabra reservada (función con retorno)        |
| ACTIO                                | <code>actio</code>                 | Palabra reservada (función sin retorno)        |
| REDDERE                              | <code>reddere</code>               | Palabra reservada (return)                     |
| NON                                  | <code>non</code>                   | Palabra reservada (negación lógica)            |
| VERUM                                | <code>verum</code>                 | Palabra reservada (literal booleano verdadero) |
| FALSUS                               | <code>falsus</code>                | Palabra reservada (literal booleano falso)     |
| NUMERUS                              | <code>numerus</code>               | Palabra reservada (tipo de dato entero)        |
| TEXTUM                               | <code>textum</code>                | Palabra reservada (tipo de dato texto)         |
| DECIMALIS                            | <code>decimalis</code>             | Palabra reservada (tipo de dato decimal)       |
| LITTERA                              | <code>littera</code>               | Palabra reservada (tipo de dato carácter)      |
| BOOL                                 | <code>bool</code>                  | Palabra reservada (tipo de dato booleano)      |
| INCREMENTO                           | <code>++</code>                    | Operador de suma abreviada                     |
| DECREMENTO                           | <code>--</code>                    | Operador de resta abreviada                    |
| MAS, MENOS, POR, DIV                 | <code>+ - * /</code>               | Operadores aritméticos                         |
| IGUALA, DIFERENTE                    | <code>== !=</code>                 | Operadores relacionales de igualdad            |
| MENOR, MAYOR, MENORIGUAL, MAYORIGUAL | <code>&lt; &gt; &lt;= &gt;=</code> | Operadores relacionales de orden               |
| AND, OR                              | <code>&amp;&amp;   </code>         | Operadores lógicos                             |
| IGUAL                                | <code>=</code>                     | Operador de asignación                         |

|                      |                         |                               |
|----------------------|-------------------------|-------------------------------|
| DOSPUNTOS            | :                       | Símbolo (tipado de variables) |
| PUNTOCOMA            | ;                       | Símbolo (fin de instrucción)  |
| COMA                 | ,                       | Símbolo (separador)           |
| PUNTO                | .                       | Símbolo (acceso a atributo)   |
| LPAREN, RPAREN       | ( )                     | Símbolos de agrupación        |
| LLAVE, RLLAVE        | { }                     | Símbolos de bloque            |
| LCORCHETE, RCORCHETE | [ ]                     | Símbolos de arreglo/índice    |
| LECTURA              | <<                      | Operador de entrada           |
| ESCRITURA            | >>                      | Operador de salida            |
| NUMERO               | [0-9]+                  | Literal numérico entero       |
| DECIMAL              | [0-9]+ '.'<br>[0-9]+    | Literal numérico decimal      |
| CADENA               | "..."                   | Literal de cadena de texto    |
| CARACTER             | '.'                     | Literal de carácter           |
| ID                   | [a-zA-Z_] [a-zA-Z0-9_]* | Identificador                 |

### 3.2 Manejo de espacios, comentarios y su justificación

```
WS : [ \t\r\n]+ -> channel(HIDDEN);
COMENTARIO_LINEA : '//' ~[\r\n]* -> channel(HIDDEN);
COMENTARIO_BLOQUE : '##' .*? '##' -> channel(HIDDEN);
```

Los espacios en blanco y los comentarios (de línea `//` y de bloque `## ##`) se envían al canal `HIDDEN` de ANTLR en lugar de descartarse con la directiva `skip`. Esta decisión de diseño tiene una razón concreta: `CommonTokenStream` solo entrega al analizador sintáctico los tokens del canal por defecto, por lo que el comportamiento del parser es idéntico al que tendría con `skip`. Sin embargo, al mantenerlos en el canal `HIDDEN`, estos tokens siguen existiendo y son recuperables mediante `lexer.nextToken()`, lo cual es indispensable para el módulo de resaltado de sintaxis en tiempo real, que necesita poder identificar y colorear los comentarios sin alterar el comportamiento del análisis sintáctico.

## Detección y manejo de errores léxicos

Cuando el lexer generado por ANTLR encuentra un carácter que no corresponde a ningún patrón definido en la gramática (por ejemplo, \$, @ u otro símbolo no contemplado), internamente genera una `LexerNoViableAltException` y recupera el análisis saltando el carácter inválido para continuar tokenizando el resto de la entrada.

Para capturar estos errores, se registró un `BaseErrorListener` personalizado directamente sobre la instancia del lexer (no del parser), dentro del método `Compilador.tokenizar()`. Este listener intercepta cada `LexerNoViableAltException`, extrae la posición del carácter inválido (`getStartIndex()`) y la almacena en una lista de errores léxicos independiente de la lista de tokens válidos. Esta información se utiliza posteriormente para resaltar visualmente en el editor los caracteres no reconocidos por el lenguaje, sin necesidad de ejecutar el análisis sintáctico completo.

## Análisis Sintáctico

### Generación del parser con ANTLR4

El analizador sintáctico (`CodexLatinusParser`) se generó automáticamente por ANTLR4 a partir de las reglas de la gramática definidas en `CodexLatinus.g4`. A partir del stream de tokens producido por el lexer (`CommonTokenStream`), el parser construye un árbol de parseo (*parse tree*) siguiendo un análisis descendente predictivo (LL), punto de entrada mediante la regla inicial programa.

### Reglas principales de la gramática

La estructura general de un programa en `CodexLatinus` se compone de tres secciones opcionales/obligatorias, procesadas en el siguiente orden:

- `seccionVariables (VARIABLES>)`: declaración de variables y arreglos globales, y definición de estructuras.
- `seccionFunciones (MUNERA>)`: definición de funciones con o sin valor de retorno.
- `seccionPrincipal (MAIOR> ... FINIS;)`: bloque principal de instrucciones ejecutables, obligatorio y siempre presente.

Dentro de estas secciones, las reglas se organizan en tres grandes grupos:

- Declaraciones: variables (`declaracionVariable`), arreglos (`declaracionArray`), estructuras (`declaracionEstructuraDef`, `declaracionEstructuraUso`).
- Instrucciones: asignaciones, llamadas a función, condicionales (`si/aliter/finis`), ciclos (`dum`, `facere...dum`, `per`), lectura/escritura, retorno, `perge/interrumpe`.

- Expresiones: organizadas en una jerarquía de precedencia descendente típica de un parser LL - expresionOr -> expresionAnd -> expresionIgualdad -> expresionRelacional -> expresionAditiva -> expresionMultiplicativa -> expresionUnaria -> expresionPostfija -> expresionPrimaria, garantizando así la correcta precedencia y asociatividad de los operadores lógicos, relacionales y aritméticos.

La gramática completa, con todas sus reglas, se encuentra documentada íntegramente en la sección 8 de este manual.

## Manejo de errores sintácticos

De forma análoga al manejo de errores léxicos, el análisis sintáctico utiliza un `ErrorListener` personalizado (`com.compi2.codexlatinus.errores.ErrorListener`), registrado sobre la instancia del parser en `Compilador.analizar()` mediante `parser.removeErrorListeners()` y `parser.addErrorListener(el)`. Este listener sobrescribe el comportamiento por defecto de ANTLR (que solo imprime los errores en consola), capturando cada error sintáctico reportado durante el parseo y transformándolo en una instancia de la clase `Error`, la cual almacena el mensaje descriptivo y la ubicación (línea y columna) del error. Estos errores se agregan a la lista `resultado.errores`, que finalmente se muestra al usuario en la interfaz gráfica (`listaErrores`) tras ejecutar el análisis completo.

## Pila de análisis sintáctico

Como apoyo didáctico para visualizar el funcionamiento interno del analizador sintáctico, el sistema incluye un módulo (`GeneradorPila`) que registra la evolución de la pila de análisis durante el proceso de parseo, generando una secuencia de instantáneas (`EstadoPila`) que representan el contenido de la pila y la entrada restante en cada paso. Esta secuencia de estados puede visualizarse posteriormente en una ventana dedicada (`PilaController`), permitiendo al usuario observar paso a paso cómo el parser va reduciendo y desplazando símbolos conforme reconoce la estructura sintáctica del programa.

# Árbol de Sintaxis Abstracta (AST)

## Diseño de nodos

El AST se modela mediante una jerarquía de clases con `NodoAst` como clase base abstracta, de la cual heredan dos grandes familias de nodos: las **instrucciones** (paquete `ast.instrucciones`) y las **expresiones** (paquete `ast.expresiones`). Cada tipo de construcción sintáctica del lenguaje (declaración, condicional, ciclo, operación, literal, etc.) cuenta con su propia clase, lo que permite representar el programa de forma estructurada y desacoplada del árbol de parseo generado por ANTLR. La

clase Ast actúa como contenedor raíz, agrupando las tres secciones del programa (variables, funciones e instrucciones principales) una vez construidas.

## **Construcción del AST**

La construcción del AST se realiza mediante el patrón *Visitor*, aplicando el recorrido del árbol de parseo generado por ANTLR (ProgramaContext y sus derivados) a través de la clase AstVisitor, que extiende el visitor base generado automáticamente por ANTLR (CodexLatinusBaseVisitor). Por cada regla gramatical visitada, AstVisitor instancia el nodo NodoAst correspondiente y lo enlaza con sus nodos hijos, replicando la estructura jerárquica del programa fuente. La clase AgregarStmt colabora en este proceso, encargándose de añadir las instrucciones (statements) construidas a los bloques correspondientes (cuerpos de funciones, condicionales y ciclos). Adicionalmente, durante este mismo recorrido, BuilderTablaSimbolos construye la tabla de símbolos, registrando cada declaración de variable, función o estructura en el ámbito (Scope) correspondiente.

## **Visualización del AST**

Para fines didácticos, el sistema permite visualizar gráficamente el AST generado. La clase ArbolBuilder transforma el árbol de nodos (NodoAst) en una representación adaptada para su despliegue visual, utilizando la clase NodoGrafico para representar cada nodo con su posición, forma y conexiones hacia sus nodos hijos. Esta representación se muestra en una ventana independiente gestionada por AstController, accesible desde el menú principal del editor (verAst() en MainController).

## **Análisis Semántico**

### **Tabla de símbolos**

La tabla de símbolos (TablaSimbolos) almacena todos los identificadores declarados en el programa (variables, arreglos, funciones y estructuras), organizados jerárquicamente por ámbito mediante la clase Scope. Cada símbolo se representa con una instancia de Simbolo, que almacena su nombre, tipo (TipoEnum/Structura) y clasificación (TipoSimbolo: variable, función, parámetro o estructura). Esta organización por ámbitos permite resolver correctamente el alcance (scope) de cada identificador, distinguiendo entre variables globales, variables locales de función y variables declaradas dentro de bloques de control.

### **Reglas semánticas validadas**

El analizador semántico valida, entre otras, las siguientes reglas sobre el AST ya construido:

- Verificación de tipos en asignaciones y expresiones (compatibilidad entre el tipo declarado y el tipo del valor asignado).
- Detección de variables, funciones o estructuras utilizadas sin haber sido declaradas previamente.
- Detección de declaraciones duplicadas dentro del mismo ámbito.
- Validación de tipos de retorno en funciones (ratio) respecto al tipo declarado.
- Validación de la correcta asignación de atributos al instanciar una estructura (declaracionEstructuraUso).
- Verificación de tipos en condiciones de estructuras de control (si, dum, facere...dum, per), las cuales deben evaluar a un valor booleano.

## Flujo de AnalizadorSemantico

AnalizadorSemantico recibe el AST construido y la tabla de símbolos, y recorre el árbol validando cada nodo según las reglas anteriores. Los errores detectados se acumulan en una lista interna (getErrores()), que posteriormente se integra a la lista general de errores en Compilador.analizar(), junto con los errores léxicos y sintácticos ya detectados en fases anteriores.

## Funcionalidades Adicionales

### Traductor a Pig Latin

TraductorPigLatin implementa una funcionalidad complementaria del proyecto, que traduce el código fuente (o los identificadores del programa) al idioma inventado PigLatin. El resultado se visualiza en una ventana independiente gestionada por PigLatinController, accesible desde el menú principal (verPigLatin() en MainController).

### Resaltado de sintaxis en tiempo real

#### Justificación técnica

El coloreado de sintaxis se implementó reutilizando directamente el analizador léxico generado por ANTLR (CodexLatinusLexer), en lugar de emplear listas de palabras clave independientes o expresiones regulares propias desconectadas del compilador. Esta decisión garantiza que el resaltado visual refleje exactamente las mismas reglas léxicas que rigen el análisis real del lenguaje, evitando inconsistencias entre lo que el usuario ve coloreado y lo que el compilador efectivamente reconoce como válido.

### Flujo del resaltado

CodeArea (RichTextFX)

```

multiPlainChanges().successionEnds(100ms)
.
Resaltador.calcularEstilos(texto)
.
Compilador.tokenizar(texto)
(instancia CodexLatinusLexer, recorre con nextToken())
.
Lista de tokens + lista de errores léxicos
.
Resaltador clasifica cada token por tipo → clase CSS
.
StyleSpans<Collection<String>>
.
codeArea.setStyleSpans(0, estilos)

```

Cada vez que el usuario deja de escribir durante 100 milisegundos, el CodeArea dispara el recálculo de estilos. Resaltador invoca a Compilador.tokenizar(), que ejecuta únicamente la fase léxica (sin construir el árbol de parseo ni ejecutar el análisis semántico), clasifica cada token según su tipo (palabra clave, tipo de dato, cadena, número o comentario) y construye los StyleSpans correspondientes, que finalmente se aplican al área de texto.

## Interfaz Gráfica (GUI)

### Vista principal

MainController gestiona la ventana principal del editor, la cual integra un CodeArea de RichTextFX con numeración de líneas (LineNumberFactory) y resaltado de sintaxis en tiempo real. Desde esta vista el usuario puede crear, abrir y guardar archivos .lat, ejecutar el análisis completo del código y acceder a las ventanas de visualización secundarias.

### Vistas secundarias

- **Vista AST** (AstController): despliega gráficamente el árbol de sintaxis abstracta generado.
- **Vista Tabla de Símbolos** (TablaSimboloController): despliega los símbolos declarados, organizados por ámbito.
- **Vista Pila de Análisis** (PilaController): despliega la evolución de la pila del parser durante el análisis sintáctico.
- **Vista PigLatin** (PigLatinController): despliega la traducción del código a PigLatin.

## Estilos y hoja CSS del editor

El editor utiliza una hoja de estilos CSS dedicada que define un tema oscuro para el área de código, con clases específicas para cada categoría léxica (.palabra-clave, .tipo-dato, .cadena, .numero, .comentario) y para la numeración de líneas (.lineno). Dado que RichTextFX aplica los estilos al nodo Text interno de cada segmento, las reglas de resaltado se definieron con selectores de especificidad suficiente (.code-area .text.<clase>) para prevalecer sobre el estilo base del texto (.code-area .text).

## Requisitos de software

- JDK 26
- Demás requisitos descritos en POM del proyecto

## Conclusiones y Trabajo Futuro

El desarrollo de CodexLatinus permitió aplicar de forma práctica las fases fundamentales de un compilador -análisis léxico, sintáctico y semántico- utilizando herramientas estándar de la industria como ANTLR4, integradas en una aplicación de escritorio funcional con JavaFX. La integración del resaltado de sintaxis directamente sobre el analizador léxico real del compilador, en lugar de mecanismos independientes, refuerza la coherencia entre lo que el usuario visualiza y lo que el sistema efectivamente reconoce como válido.

Como trabajo futuro, el proyecto podría extenderse incorporando una fase de generación de código intermedio o ejecución (intérprete), así como validaciones semánticas adicionales y mejoras en la retroalimentación visual de errores dentro del propio editor.

## Referencias

- Documentación oficial de ANTLR4: <https://www.antlr.org/>
- Documentación de RichTextFX: <https://github.com/FXMisc/RichTextFX>
- Documentación oficial de JavaFX: <https://openjfx.io/>
- Link a Github: <https://github.com/MonseGomezR/practical-compi2-ss2026>